

1 a.) Why this architecture category fits our design

Chosen style: **Layered Architecture (3–4 layers) + separate AI service**

Presentation Layer (Frontend – React)

Components:

AIChatBox, TicketForm, TicketList, Login, AdminDashboard, etc.

Responsibility:

Render UI, capture user input, call backend APIs, show AI responses and ticket data.

Granularity:

Fine-grained React components grouped into pages/views, all talking to backend via REST.

Application / API Layer (Spring Boot – REST controllers)

Components:

AuthController, TicketController, MessageController, AdminController, AIProxyController, etc.

Responsibility:

Expose REST API, validate requests, orchestrate calls to services, convert DTOs.

Granularity:

Each controller groups related endpoints (auth, tickets, AI integration) as coarse-grained services.

Domain / Service Layer (Spring Boot services)

Components:

AuthService, TicketService, MessageService, NotificationService, AnalyticsService, KnowledgeBaseService.

Responsibility:

Implement business rules: ticket lifecycle, escalation, AI confidence routing, notification triggers.

Granularity:

Each service encapsulates one major business capability (auth, ticketing, messaging, analytics).

Data Access Layer (Repositories + Data Stores)

Components:

JPA repositories: UserRepository, TicketRepository, MessageRepository, DocumentRepository, NotificationRepository.

Data stores: User DB, Ticket DB, Message DB, Knowledge Base metadata, Analytics tables.

Responsibility:

CRUD and queries; mapping between entities and database.

Granularity:

One repository per aggregate root (User, Ticket, Message), giving clear separation of concerns.

External AI Service (Python FastAPI)

Components:

RAGEngine, DocumentProcessor, FastAPI endpoints: /api/ai/query, /api/ai/categorize, /api/ai/index-document.

Responsibility:

RAG pipeline, Gemini integration, embeddings, semantic search, confidence scoring and categorization.

Granularity:

Coarse-grained: entire AI behavior is encapsulated as one service behind a small HTTP API, consumed by the Spring AIServiceClient.

So, structurally we have classic layers inside the Spring Boot app, while the AI module is separated as its own service to isolate heavy ML logic and dependencies. This matches a layered architecture with one external service, not a fully decomposed microservices system.

1 b.) Why this is the best style for our project

a) Scalability

The layered core (auth + tickets + chat + notifications) is a typical small-to-medium web app; layered architecture is recommended for such scale because it is adequately scalable.

The AI service is the most resource-intensive part (LLM calls, vector DB, embeddings), and we can scale it independently (e.g., more CPU/GPU, separate container) while keeping the rest of the system as a single deployable Spring Boot app. This gives us targeted scalability where it matters without full microservices complexity.

If traffic grows, we can:

- Horizontally scale the Spring Boot app behind a load balancer.
- Independently scale the Python AI container (or move to GPU machines) without touching the Java code.

b) Maintainability

In, layered architecture each layer has a clear responsibility: UI, API, business logic, data access.

Changes in one layer (e.g., switching from MySQL to PostgreSQL, or updating React components) do not leak into other layers as long as interfaces remain stable.

Isolating AI logic into a separate service:

- Keeps Spring codebase clean (no heavy ML/LLM dependencies).
- Allows the AI module to evolve (new prompts, different LLM provider, new RAG strategy) without changing ticketing/business logic.

Full microservices would add deployment, monitoring and distributed-system complexity that is hard to justify.

c) Performance

For most requests (auth, ticket list, status updates), the Spring Boot monolith serves data directly from the database with low latency and no cross-service hops, which is better than a very fine-grained microservices mesh.

Only AI-related calls go over the network to the FastAPI service; that cost is dominated by LLM latency anyway, so splitting it out does not materially hurt performance.

Layered architecture avoids the overhead of network calls between many small services and complex orchestration, which can otherwise degrade performance in simpler systems.

d) Extensibility and Future Growth

Starting from a layered modular monolith is widely recommended as a good base that can later be extracted into microservices if necessary.

In the future, if we need:

- Separate “Ticket Service” or “Notification Service” as independent microservices,

The AI module is already microservice-like, so we’ve practiced the pattern on the piece that benefits from it most (RAG and LLM integration) without committing the whole system to that style.

2) Mention the components present in your software engineering project (application components).

1. Frontend Layer (React.js)

UI Components

AIChatBox - Real-time chat interface with AI assistant

TicketForm - Create new support tickets

TicketList - Display all tickets with filters

TicketDetails - View individual ticket with full chat history

LoginForm - User authentication interface

RegisterForm - New user registration

Dashboard - Main dashboard for customers/agents/admins

TicketCategorizer - Visual ticket categorization component

KnowledgeBaseUpload - Document upload interface (admin)

AnalyticsDashboard - Charts and metrics display

UserProfile - User settings and profile management

NotificationPanel - Real-time notification display

Service Components

aiService.js - API client for AI service integration

authService.js - Authentication API calls

ticketService.js - Ticket management API calls

messageService.js - Chat/messaging API calls

analyticsService.js - Analytics data fetching

2. Backend Layer (Spring Boot)

Controller Components (API Layer)

AuthController - /api/auth/* endpoints

POST /register, /login, /verify, /forgot-password

TicketController - /api/tickets/* endpoints

POST /, GET /{id}, GET /my-tickets, PUT /{id}/status, PUT /{id}/assign

MessageController - /api/messages/* endpoints

POST /, GET /ticket/{ticketId}

AdminController - /api/admin/* endpoints

User management, system configuration

AIProxyController - /api/ai-proxy/* endpoints

Proxy calls to Python AI service

Service Components (Business Logic Layer)

AuthService - User authentication and authorization logic

registerCustomer(), login(), verifyEmail(), requestPasswordReset()

TicketService - Ticket lifecycle management

createTicket(), updateStatus(), assignAgent(), autoClose()

MessageService - Chat message handling

saveMessage(), saveAiMessage(), getMessagesByTicket()

UserService - User management

createUser(), updateUser(), deleteUser(), getUsersByRole()

NotificationService - Email/SMS notifications

sendTicketCreatedEmail(), sendStatusUpdateEmail(), sendEscalationEmail()

AnalyticsService - Generate reports and metrics

getDashboardMetrics(), getTicketTrends(), generateReport()

AIServiceClient - Integration with AI service

getAiResponse(), categorizeTicket()

TicketAutomationService - Scheduled tasks

autoCloseInactiveTickets(), escalateHighPriorityTickets(), sendReminderEmails()

EmailService - SMTP email sending

sendEmail(), sendVerificationEmail(), sendPasswordResetEmail()

Repository Components (Data Access Layer)

UserRepository - User CRUD operations

CustomerRepository - Customer-specific queries

SupportAgentRepository - Agent-specific queries

AdministratorRepository - Admin-specific queries

TicketRepository - Ticket CRUD and search

MessageRepository - Message storage and retrieval

DocumentRepository - Knowledge base document metadata

NotificationRepository - Notification tracking

AnalyticsRepository - Metrics and reporting queries

AuditLogRepository - System activity logs

Entity Components (Domain Models)

User (abstract) - Base user entity

Customer - Customer user entity

SupportAgent - Agent user entity

Administrator - Admin user entity

Ticket - Support ticket entity

Message - Chat message entity

Document - Knowledge base document entity

Notification - Email/SMS notification entity

Report - Analytics report entity

AuditLog - System audit trail entity

Security Components

JwtUtil - JWT token generation and validation

JwtAuthenticationFilter - Request authentication filter

SecurityConfig - Spring Security configuration

PasswordEncoder - BCrypt password hashing

Configuration Components

CorsConfig - Cross-origin resource sharing settings

WebSocketConfig - WebSocket configuration for real-time chat

SchedulerConfig - Scheduled task configuration

EmailConfig - SMTP email configuration

3. AI Service Layer (Python FastAPI)

API Components

main.py - FastAPI application entry point

GET /health - Health check

POST /api/ai/query - Generate AI response

POST /api/ai/categorize - Categorize ticket

POST /api/ai/index-document - Index document

Core AI Components

RAGEngine - Retrieval-Augmented Generation engine

get_response() - Generate AI answers with RAG

categorize_ticket() - Auto-categorize tickets

_calculate_confidence() - Compute confidence scores

DocumentProcessor - Document handling

load_document() - Load PDF/text files

process_and_index() - Chunk and embed documents

get_vectorstore() - Access vector database

Integration Components

GoogleGenerativeAIEmbeddings - Generate embeddings via Gemini API

ChatGoogleGenerativeAI - LLM for response generation

Chroma - Vector database for semantic search

RecursiveCharacterTextSplitter - Document chunking

Data Models (Pydantic)

QueryRequest - AI query input

AIResponse - AI response output

CategorizationRequest - Ticket categorization input

CategorizationResponse - Category and priority output

4. Data Components

Relational Databases (MySQL)

User Database - User accounts, roles, credentials

Tables: users, customers, support_agents, administrators

Ticket Database - Support tickets

Tables: tickets, ticket_tags, ticket_history

Message Database - Chat messages

Tables: messages

Knowledge Base Database - Document metadata

Tables: kb_documents

Analytics Database - Metrics and logs

Tables: analytics_metrics, audit_logs, reports

Vector Database (ChromaDB)

Embeddings Store - Document embeddings for semantic search

Collections: Document chunks with metadata (doc_id, source, chunk_id)

5. External Service Components

Email Service

Gmail SMTP - Send email notifications

Used by: EmailService, NotificationService

AI/ML Service

Google Gemini API - LLM for response generation and embeddings

Used by: RAGEngine, DocumentProcessor

